IRON GYM

SUPLEMENTOS

Documentación del Punto de Venta

Aplicación de escritorio desarrollada en Python con Tkinter

Alumno:	Leonardo Damián Hernández
Carrera:	Ingeniería en Software
Grupo:	301
Materia:	Lenguajes de Programación
Actividad:	Punto de venta (Iron Gym - Suplementos)
Fecha:	27 de octubre de 2026

1. Introducción

Este documento describe el sistema de **Punto de Venta (PDV)** de la tienda ficticia **Iron Gym - Suplementos**, desarrollado como actividad de la materia de *Lenguajes de Programación* de Ingeniería en Software. El programa permite a un administrador iniciar sesión, consultar el inventario de suplementos, armar una venta, calcular el total y el cambio, descontar existencias automáticamente, consultar el historial de ventas de la sesión y generar tickets en formato de texto (TXT).

El sistema está escrito en **Python 3** con la biblioteca estándar **Tkinter** para la interfaz gráfica, **decimal** para el manejo preciso de dinero y **datetime** para el registro de fecha y hora de cada venta. Todo el código se encuentra en un único archivo (`pdv.py`).

2. Objetivo

Desarrollar una aplicación de escritorio que simule el funcionamiento básico de un punto de venta, aplicando conceptos de programación como funciones, clases, estructuras de datos (diccionarios, listas, tuplas), manejo de excepciones, validación de datos y programación orientada a eventos.

3. Tecnologías y módulos utilizados

Módulo / biblioteca	Uso dentro del programa
`tkinter`	Ventanas, etiquetas, botones, campos de texto, Spinbox, menú y cuadros de diálogo.
`tkinter.ttk`	Widget Treeview para mostrar las tablas de inventario, carrito e historial.
`tkinter.messagebox`	Mensajes de error, advertencia y confirmación al usuario.
`tkinter.filedialog`	Cuadro «Guardar como» para exportar el ticket.
`decimal (Decimal)`	Conversión y validación exacta de los montos de pago.
`datetime`	Fecha y hora de cada venta (formato AAAA-MM-DD HH:MM:SS).

4. Estructura del programa

El código se divide en funciones auxiliares, una clase que simula el almacenamiento de datos y una clase que construye la interfaz gráfica. Esta separación mantiene la lógica del negocio independiente de la interfaz.

4.1 Datos iniciales: DEFAULT_PRODUCTS

Es una tupla de tuplas con la clave, nombre, precio (en pesos) y existencia inicial de cada producto. El catálogo base contiene siete productos con 10 piezas cada uno:

Producto	Precio	Existencia inicial
Whey Protein	$850.00	10
Isolate Protein	$1,100.00	10
Mass Gainer	$750.00	10
Creatina	$600.00	10
Omega-3	$350.00	10
Pre-entreno	$500.00	10
Vaso Shaker	$150.00	10

4.2 Funciones auxiliares

- **parse_money_to_cents(value)**: convierte el texto escrito por el usuario (por ejemplo «$3,000», «1000.50» o «1.000,50») en centavos enteros. Elimina el símbolo $ y los espacios, interpreta comas y puntos como separadores, y lanza `ValueError` si el campo está vacío, el monto es negativo, no es numérico o tiene más de dos decimales.
- **format_money(cents)**: convierte centavos a texto con formato de moneda, por ejemplo `$1,400.00`.
- **format_ticket(sale)**: genera el texto del ticket de una venta con encabezado, folio, fecha, cliente, renglones de productos, total, monto recibido y cambio.

4.3 Clase LocalStore (almacén de datos en memoria)

Simula una base de datos. Guarda los productos en un diccionario, las ventas en una lista y un contador `next_sale_id` para asignar folios consecutivos. Sus métodos son:

- **authenticate_admin(usuario, contraseña)**: devuelve verdadero solo si el usuario es «admin» y la contraseña es «admin».
- **get_inventory()**: devuelve copias de los productos ordenadas alfabéticamente por nombre.
- **update_stock(clave, cantidad)**: cambia la existencia de un producto; rechaza cantidades negativas o productos inexistentes.
- **record_sale(cliente, carrito, pagado)**: valida el nombre del cliente, que el carrito no esté vacío, que haya existencias suficientes y que el pago cubra el total. Si todo es correcto descuenta el inventario, calcula el cambio, crea el registro de la venta con fecha y hora, y le asigna un folio.

- **get_sales()** y **get_sale(id)**: consultan el historial (más recientes primero) o una venta específica.

4.4 Clase PuntoDeVenta (interfaz gráfica)

Hereda el control de la ventana principal de Tkinter y define la paleta de colores del sistema (fondo `#1e1e1e`, paneles `#2b2b2b` y acento naranja `#ff4500`). Contiene el carrito de compras (diccionario clave → cantidad) y la última venta realizada. Sus métodos principales son:

- **show_login()**: muestra la pantalla de inicio de sesión.
- **on_inventory_select()**, **refresh_inventory()**: seleccionan y refrescan la tabla de inventario.
- **add_to_cart()**, **remove_from_cart()**, **refresh_cart()**: administran el carrito y recalculan el total.
- **update_change_hint()**: calcula el cambio en tiempo real mientras se escribe el pago.
- **update_stock()**: modifica manualmente la existencia del producto seleccionado.
- **checkout()**: cobra la venta, limpia el formulario y ofrece guardar el ticket.
- **save_ticket()**, **save_last_ticket()**: guardan el ticket en un archivo TXT.
- **show_sales_history()**: abre una ventana con el historial de ventas de la sesión.
- **logout()**: vacía el carrito y regresa al inicio de sesión.

5. Funcionamiento del sistema (capturas de pantalla)

5.1 Inicio de sesión

Al ejecutar el programa se muestra la pantalla de acceso con el título **IRON GYM - SUPLEMENTOS**, los campos *Usuario* y *Contraseña* y el botón *Iniciar sesión*. El sistema indica las credenciales de acceso (admin / admin). Solo el administrador puede entrar al punto de venta.

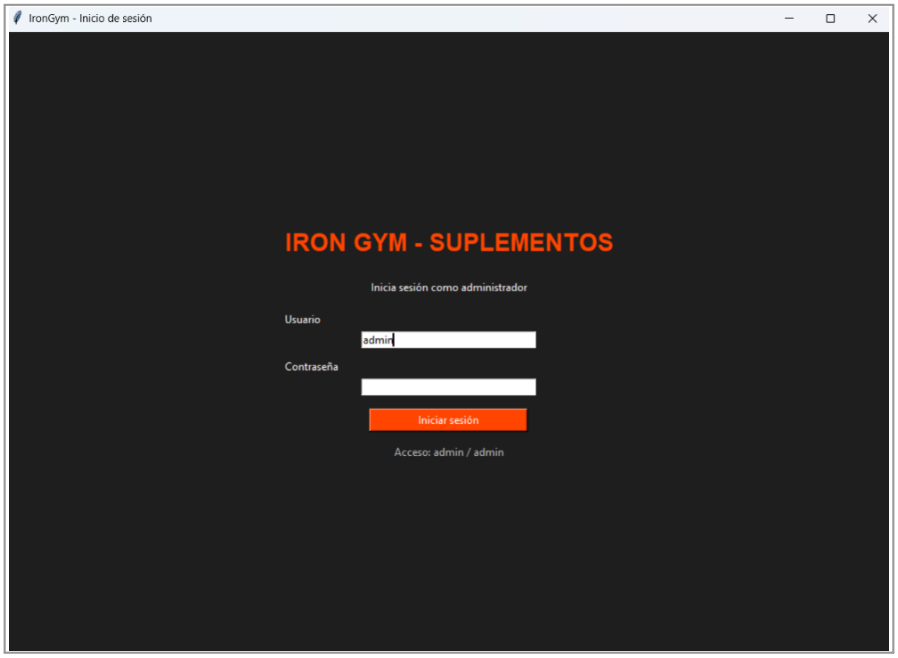


Figura 1. Pantalla de inicio de sesión.

Si el usuario o la contraseña son incorrectos, el sistema muestra un mensaje de error y no permite el acceso.

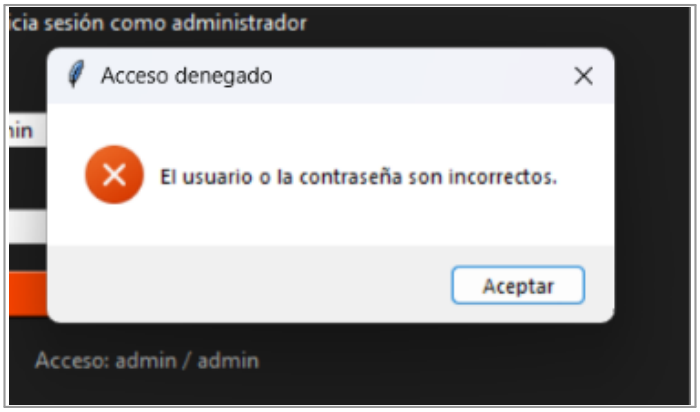


Figura 2. Mensaje «Acceso denegado» ante credenciales incorrectas.

5.2 Pantalla principal del punto de venta

Tras iniciar sesión se muestra la ventana principal, dividida en dos paneles. A la izquierda está el **Inventario** con producto, precio y existencias; debajo, el control *Cantidad a vender*, el botón *Agregar al*

carrito y el campo para actualizar la existencia del producto seleccionado. A la derecha está la **Venta actual**, con el carrito, el botón *Quitar producto seleccionado*, los campos *Nombre del cliente* y *¿Con cuánto paga?*, el total, el cambio y el botón *Cobrar venta*. En la esquina superior derecha está el botón *Cerrar sesión*.

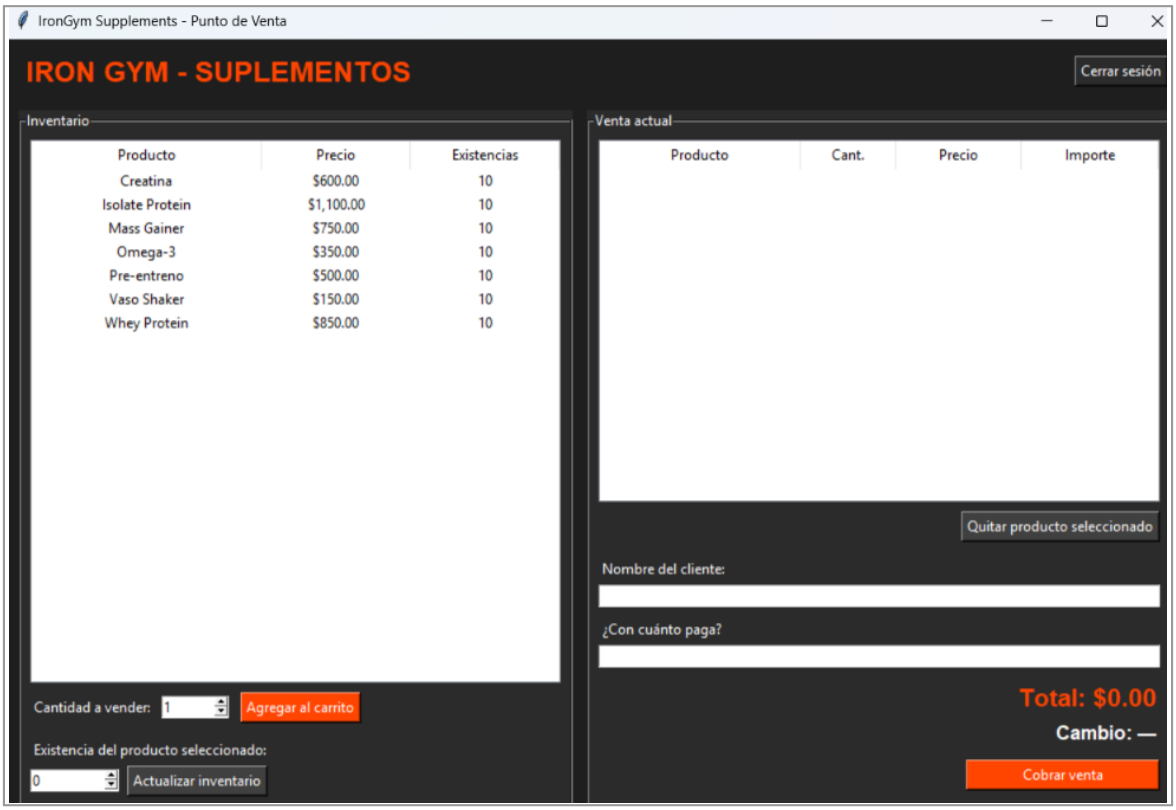


Figura 3. Pantalla principal con el carrito vacío.

5.3 Captura de una venta

Para vender se selecciona un producto en el inventario, se indica la cantidad y se pulsa *Agregar al carrito*. El sistema valida que la cantidad sea mayor a cero y que no supere las existencias. En el ejemplo se agregaron 4 piezas de **Omega-3** a $350.00 (importe $1,400.00), el cliente es **Yurem** y paga con $3,000. El cambio ($1,600.00) se calcula automáticamente al escribir el pago. En la parte inferior aparecen los botones *Historial de ventas* y *Guardar ticket de la última venta*.

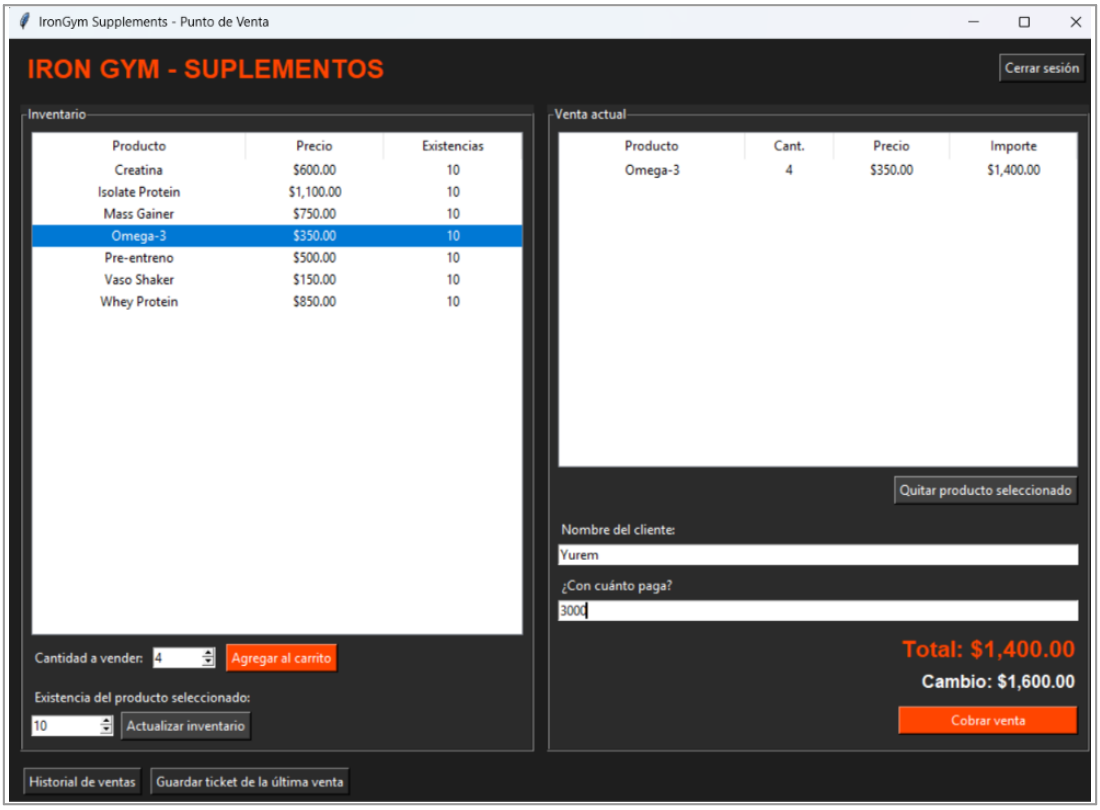


Figura 4. Venta en captura: 4 piezas de Omega-3, cliente Yurem, pago de $3,000.

5.4 Confirmación de la venta

Al pulsar *Cobrar venta*, el sistema valida los datos, descuenta el inventario, registra la venta con folio consecutivo y muestra un resumen con el total, el pago y el cambio. Además pregunta si se desea guardar el ticket como archivo TXT.

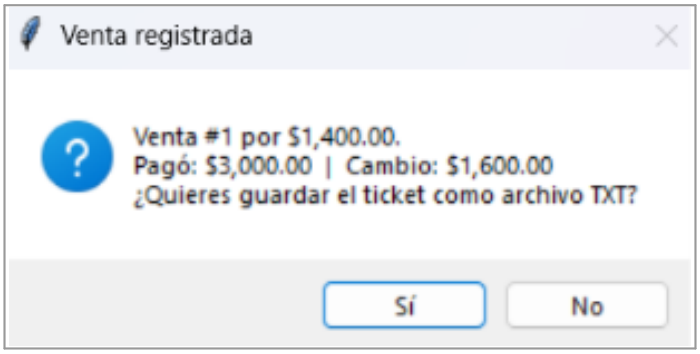


Figura 5. Mensaje «Venta registrada» con la opción de guardar el ticket.

5.5 Generación del ticket

Si el usuario elige «Sí», se abre el cuadro de diálogo del sistema operativo para guardar el archivo. El nombre sugerido es `ticket_N.txt`, donde N es el folio de la venta.

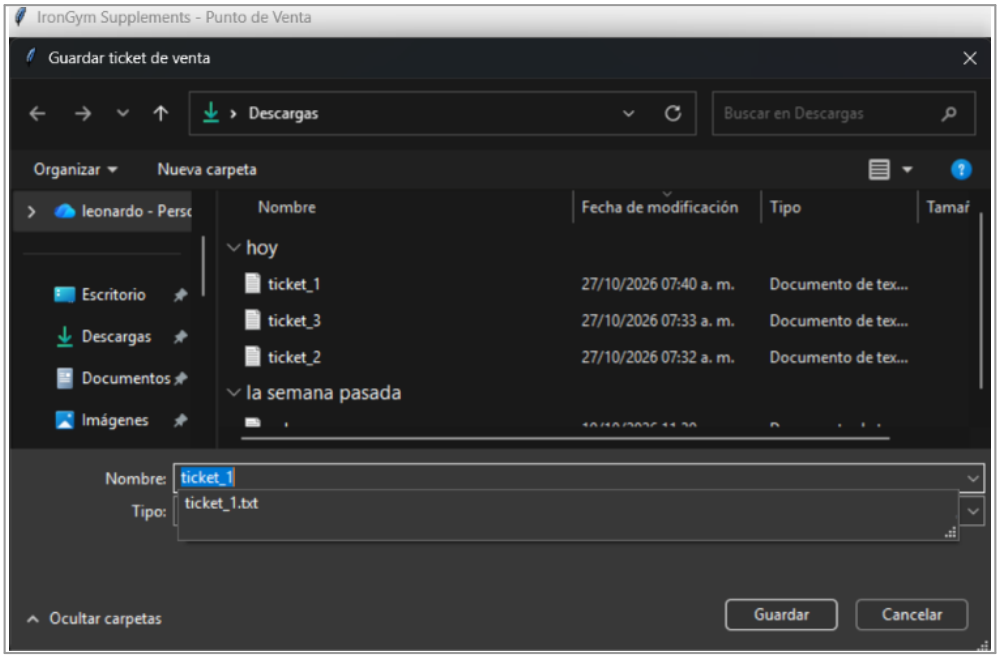


Figura 6. Cuadro de diálogo «Guardar ticket de venta».

El archivo generado contiene el nombre de la tienda, el folio, la fecha y hora, el cliente, el detalle de cada producto con su cantidad, precio unitario e importe, y al final el total, el monto recibido y el cambio.

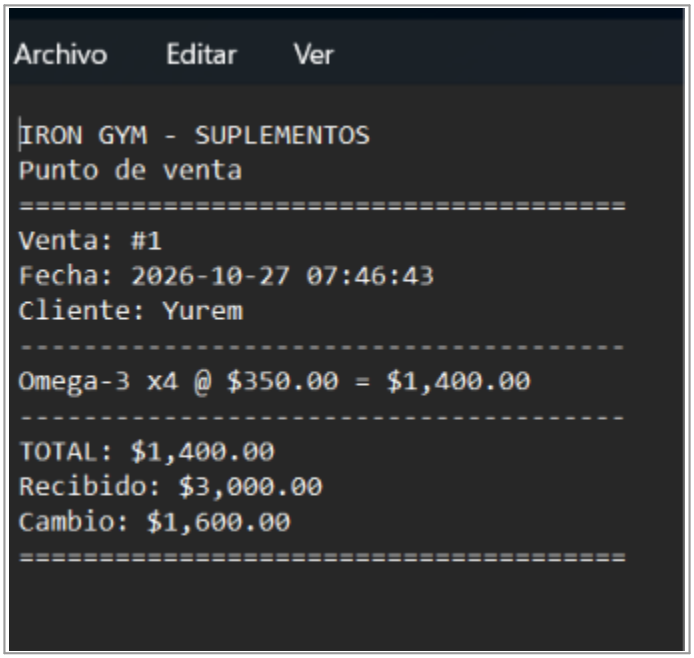


Figura 7. Ticket de la venta #1 abierto en el Bloc de notas.

5.6 Historial de ventas

El botón *Historial de ventas* abre una ventana con una tabla que muestra el folio, la fecha, el cliente y el total de cada venta realizada durante la sesión. Desde ahí se puede seleccionar una venta y volver a guardar su ticket con el botón *Guardar ticket seleccionado como TXT*.

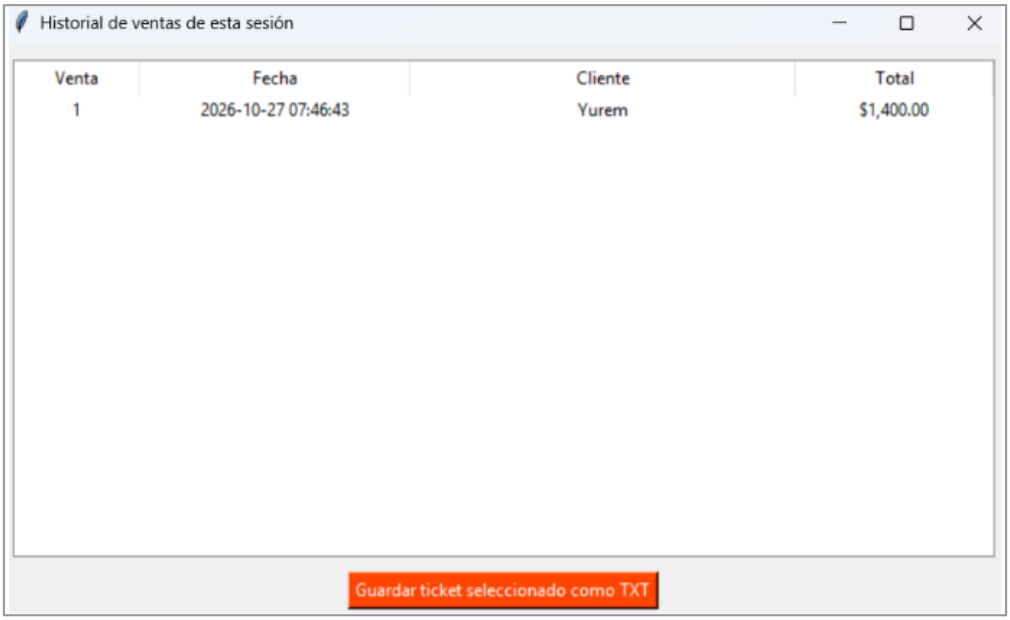


Figura 8. Ventana «Historial de ventas de esta sesión».

6. Validaciones y manejo de errores

El programa evita datos incorrectos mediante validaciones y excepciones `ValueError`, mostrando al usuario mensajes claros con `messagebox`:

Situación	Respuesta del sistema
Usuario o contraseña incorrectos	Mensaje «Acceso denegado».
Cantidad menor a 1 o no numérica	Error «Cantidad inválida».
Cantidad mayor a las existencias	Advertencia «Existencias insuficientes» con las piezas disponibles.
Cobrar sin nombre de cliente	«Escribe el nombre del cliente.»
Cobrar con el carrito vacío	«Agrega al menos un producto a la venta.»
Pago vacío, negativo o con más de 2 decimales	Mensaje indicando el formato correcto del monto.
Pago menor al total	Indica que el pago no cubre el total; en pantalla se muestra «pago insuficiente».
Existencia negativa al actualizar inventario	«La existencia no puede ser negativa.»
Error al escribir el archivo del ticket	Mensaje «Error al guardar ticket».

7. Fragmentos de código relevantes

Autenticación del administrador

```
def authenticate_admin(self, username, password):
    return username.strip() == "admin" and password == "admin"
```

Cálculo del cambio al registrar una venta

```
total_cents = total * 100
if paid_cents < total_cents:
    raise ValueError("El pago no cubre el total")
...
"change_cents": paid_cents - total_cents,
```

Descuento de existencias

```
for item in items:
    self.products[item["product_key"]]["quantity"] -= item["quantity"]
```

8. Limitaciones y posibles mejoras

- Los datos se guardan solo en memoria: al cerrar el programa se pierden el historial y los cambios de inventario. Podría usarse SQLite o archivos para almacenarlos de forma permanente.
- Hay un único usuario con contraseña fija; se podrían agregar varios usuarios con contraseñas cifradas y distintos roles.
- Solo se exportan tickets en TXT; se podría agregar exportación a PDF o impresión directa.
- Se podría añadir alta y baja de productos, descuentos y reportes de ventas por día.

9. Conclusiones

El punto de venta de Iron Gym cumple con las funciones básicas de un sistema comercial: control de acceso, consulta y actualización de inventario, captura de ventas, cálculo de cambio, emisión de tickets e historial. Su desarrollo permitió aplicar los conceptos de la materia de Lenguajes de Programación: programación orientada a objetos, funciones, estructuras de datos, manejo de excepciones, eventos e interfaces gráficas, además de separar la lógica del negocio de la interfaz de usuario.

COMPLEMENTO DEL REPORTE DE PRÁCTICA

Punto de Venta Iron Gym - Suplementos | Lenguajes de Programación | Python 3

Este complemento conserva las páginas del documento original y agrega los resultados, la discusión, el cuestionario contestado y las referencias solicitadas. Las respuestas sobre la aplicación se basan en las funciones y el comportamiento descritos en el reporte.

c) Resultados

Se documentó una aplicación de escritorio en Python 3 con Tkinter para administrar un punto de venta de suplementos. La interfaz permite iniciar sesión como administrador, consultar y actualizar existencias, armar un carrito, registrar ventas, calcular el cambio, revisar el historial de la sesión y guardar tickets en archivos TXT. Incluye validaciones para evitar cantidades inválidas, ventas sin cliente o carrito, pagos insuficientes y existencias negativas.

En el ejemplo presentado se vendieron cuatro piezas de Omega-3 a $350.00 cada una: total de $1,400.00, pago de $3,000.00 y cambio de $1,600.00. Al confirmar la operación se descuenta el inventario, se asigna un folio y se registra la fecha y hora.

d) Discusión

La práctica integra programación orientada a objetos, funciones, estructuras de datos, excepciones e interacción por eventos. Separar el almacén en memoria (LocalStore) de la interfaz (PuntoDeVenta) distingue las reglas de negocio de la presentación. Decimal y la conversión a centavos evitan errores comunes de precisión en los importes monetarios.

El alcance es didáctico: los datos y el historial se pierden al cerrar la aplicación, la cuenta de administrador está fija y los tickets solo se exportan como texto. Como mejoras, el reporte propone persistencia con SQLite o archivos, usuarios con credenciales protegidas y roles, y opciones adicionales de exportación o análisis de ventas.

e) Cuestionario

1. ¿Qué paradigmas de programación utiliza Python?

Python es multiparadigma: admite programación imperativa y procedural (instrucciones y funciones), orientada a objetos (clases y objetos) y funcional (funciones de orden superior, expresiones lambda e iteradores). En esta práctica predominan el enfoque orientado a objetos y las funciones auxiliares; Tkinter organiza la interacción mediante eventos.

2. ¿Cuál es la clasificación de los tipos de datos estándar en Python?

Tipos incorporados: numéricos (int, float, complex y bool); secuencias (str, list, tuple y range); mapeos (dict); conjuntos (set y frozenset); binarios (bytes, bytearray y memoryview); y el tipo nulo NoneType (valor None). Algunos son mutables, como list, dict y set; otros son inmutables, como int, str, tuple y frozenset.

3. ¿Cuáles son las palabras reservadas en Python? Escribe el listado y una breve descripción.

En Python 3.13, las palabras clave reservadas son: **False** (falso), **None** (ausencia de valor), **True** (verdadero); **and** (conjunción), **or** (disyunción), **not** (negación), **is** (identidad) e **in** (pertenencia); **if** (condición), **elif** (condición alternativa) y **else** (alternativa final); **for** (bucle sobre elementos) y **while** (bucle condicional); **break** (termina un bucle), **continue** (pasa a la iteración siguiente) y **pass** (instrucción vacía); **def** (define función), **return** (devuelve resultado), **lambda** (función anónima), **yield** (produce un valor de generador); **class** (define clase); **import** y **from** (importan módulos o nombres); **as** (alias o nombre en excepciones); **try**, **except**, **finally** y **raise** (manejo y lanzamiento de excepciones); **assert** (comprobación); **del** (elimina una referencia o elemento); **global** y **nonlocal** (declaran el ámbito de una variable); **with** (gestiona un contexto); **async** y **await** (definen y esperan operaciones asíncronas).

Palabras clave suaves de Python 3.13, especiales solo en contextos concretos: **match** y **case** (sentencia de patrones), **type** (sentencia de alias de tipo) y **_** (comodín en patrones).

4. ¿Qué operadores de asignación se pueden utilizar en Python?

La asignación simple es **=**. Las asignaciones aumentadas son **+=**, **-=**, ***=**, **/=**, **//=**, **%=**, ****=**, **@=**, **&=**, **|=**, **^=**, **>>=** y **<<=**; actualizan una variable aplicando una operación. **:=** es asignación como expresión (“walrus”). También se permiten asignación múltiple y desempaquetado, por ejemplo **a, b = 1, 2**, y desempaquetado extendido con *****.

5. ¿Cómo se realizan los comentarios de línea y multilinea en Python?

Un comentario de línea comienza con **#** y continúa hasta el final de esa línea. Para comentar varias líneas se coloca **#** al inicio de cada una. Las cadenas con comillas triples (''' o """) no son comentarios: pueden documentar módulos, clases y funciones como docstrings, o ser cadenas multilínea si están asignadas.

6. ¿Qué es una lista en Python y describe los métodos que utiliza para la lista?

Una lista (list) es una secuencia ordenada y mutable que permite elementos repetidos. Métodos principales: **append(x)** agrega al final; **extend(iterable)** agrega varios elementos; **insert(i, x)** inserta en una posición; **remove(x)** elimina la primera coincidencia; **pop(i)** elimina y devuelve un elemento; **clear()** vacía la lista; **index(x)** devuelve la posición de una coincidencia; **count(x)** cuenta coincidencias; **sort()** ordena; **reverse()** invierte el orden; y **copy()** crea una copia superficial.

7. ¿Qué es una tupla y cuáles son los métodos que se utilizan en Python para las tuplas?

Una tupla (tuple) es una secuencia ordenada e inmutable; puede contener valores repetidos y de distintos tipos. Sus métodos son **count(x)**, que cuenta las apariciones, e **index(x)**, que devuelve el índice de la primera aparición (o genera ValueError si no existe). Al ser inmutable, no tiene métodos para agregar, quitar o modificar elementos.

8. ¿Qué son los conjuntos en Python y cuáles son los métodos que se utilizan para los conjuntos?

Un conjunto (set) es una colección mutable, no ordenada y sin elementos duplicados; **frozenset** es la variante inmutable. Métodos de set: **add(x)** agrega; **update(it)** agrega elementos de iterables; **remove(x)** elimina o produce KeyError; **discard(x)** elimina sin error si falta; **pop()** quita y devuelve un elemento arbitrario; **clear()** vacía; **copy()** copia; **union()** combina; **intersection()** obtiene comunes; **difference()** obtiene los exclusivos del conjunto; **symmetric_difference()** obtiene los no comunes; sus variantes **update**, **intersection_update**, **difference_update** y **symmetric_difference_update** modifican el conjunto. **issubset()**, **issuperset()** e **isdisjoint()** comprueban relaciones.

9. ¿Qué estructuras de control utilizaste en el desarrollo de la práctica?

Según el código y el funcionamiento descritos, se usan condicionales **if** para validar acceso, existencias y pago; bucles **for** para recorrer los artículos de una venta y actualizar inventario; y manejo de excepciones **try/except** con ValueError para informar entradas inválidas. Tkinter también responde a eventos, por ejemplo, clics en botones. No se afirma el uso de while porque no está documentado en los fragmentos proporcionados.

10. Si utilizaste alguna de las funciones de Python, describe el procedimiento realizado para utilizarla.

La aplicación define funciones auxiliares: **parse_money_to_cents(value)** recibe el texto de pago, elimina símbolos y espacios, interpreta separadores decimales, valida que el monto sea válido y tenga como máximo dos decimales, y devuelve centavos enteros o lanza ValueError; **format_money(cents)** convierte centavos en una cadena monetaria; **format_ticket(sale)** recibe los datos de una venta y genera el texto del ticket. Durante el cobro se valida el pago, se compara con el total, se registra la venta y se calcula el cambio; luego la interfaz ofrece guardar el ticket mediante el diálogo de archivos de Tkinter. El reporte también describe métodos de las clases, como **record_sale** y **get_inventory**, que la interfaz invoca para consultar o actualizar datos.

f) Referencias bibliográficas

Covatec. (2018). *Programación en Python - nivel básico*. Disponible en: https://entrenamiento-python-basico.readthedocs.io/es/latest/index.html

López Ostenero, Fernando, y García Serrano, Ana María. (2014). *Teoría de los Lenguajes de Programación*. Editorial Universitaria Ramón Areces. ISBN 9788499611396.

Curso de Python. Lista de reproducción de YouTube. Disponible en: https://www.youtube.com/watch?v=tDYr14Ilu_4&list=PLU8oAlHdN5BlvPxziopYZRd55pdqFwkeS